

C# Value Types vs Reference Types

From Beginner → Advanced (Complete Guide)

Why this topic matters

This is one of the most misunderstood concepts in C#. Many developers memorize:

"Struct = Value Type"

"Class = Reference Type"

...but fail interviews because they don't understand **what actually happens in memory, copy behavior, performance implications, and CLR internals.**

Let's fix that.

1. What is a Value Type?

A value type **stores the actual data itself.**

When you assign it to another variable, **the entire value is copied.**

Examples:

- int
- long
- bool
- double
- decimal
- char
- DateTime
- Guid
- Enum
- Struct

CSharp

```
int x = 10;  
int y = x;  
y = 20;  
Console.WriteLine(x); // Outputs: 10  
Console.WriteLine(y); // Outputs: 20
```

A completely new copy is created.

2. What is a Reference Type?

A reference type stores a **reference (address)** to an object that lives on the managed heap.

Variables do not contain the object itself.

Examples

- class
- string
- array
- delegate
- interface
- record (class)
- object
- dynamic

CSharp

```
Person p1 = new Person();  
Person p2 = p1;  
p2.Name = "John";  
Console.WriteLine(p1.Name); // Outputs: John
```

Both variables point to the same object.

3. Visual Representation

Value Type

Stack

x -> 10

y -> 10

Independent copies

After changing y

Stack

x -> 10

y -> 20

Reference Type

Stack

```
p1 ----\
      \
      ----> Heap Object
      /
p2 ----/
```

One object.

Two references.

4. Stack vs Heap

One of the biggest misconceptions:

Value Types = Stack

Reference Types = Heap

This is **NOT** always true.

Reality:

The location depends on **where the variable is stored**, not the type itself.

Example

```
CSharp
class Employee
{
    public int Age;
}
```

Age is a value type.

But since it's inside a class, it is stored **inside the heap object**.

Similarly

```
CSharp
int[] numbers = new int[100];
```

The array object is on the **heap**.

Each int is a value type stored inside that heap allocation.

5. Copy Behavior

Value Types

Original

10

↓

Copy

10

Reference Types

Reference A ----\

\

Object

/

Reference B ----/

Only the reference gets copied.

Not the object.

6. Assignment Behavior

Value Type

```
CSharp
Point p1 = new Point(2, 3);
Point p2 = p1; // Full copy created
```

Creates a brand new copy.

Reference Type

CSharp

```
Employee e2 = e1; // Only reference copied
```

Copies only the reference.

7. Parameter Passing

Everything in C# is passed **by value by default**.

Yes...

Even reference types.

Example

CSharp

```
void Change(Person p)
{
    p.Name = "Alice"; // Affects original object
}
```

The reference itself is copied.

Both references still point to the same object.

Replacing the object

CSharp

```
void Change(Person p)
{
    p = new Person(); // Only changes local reference
}
```

Only the copied reference changes.

The caller remains unchanged.

8. ref, out and in

ref

Passes the actual variable.

No copy.

```
CSharp  
Increment(ref x);
```

out

Must assign before returning.

Used for multiple outputs.

in

Read-only reference.

Avoids copying large structs.

```
CSharp  
void Print(in LargeStruct value)  
{  
    // value is read-only  
}
```

Useful for performance.

9. Boxing

One of the most important interview questions.

CSharp

```
int number = 5;  
object obj = number; // Boxing occurs here
```

The CLR creates a new heap object.

Stack

number -> 5

Heap

Object
5

This is Boxing.

10. Unboxing

CSharp

```
int x = (int)obj; // Unboxing occurs here
```

CLR copies the value back.

Invalid cast throws exception.

11. Hidden Cost of Boxing

Every boxing operation

- Heap allocation
- Copy
- GC pressure
- Performance overhead

Avoid unnecessary boxing in hot paths.

12. Struct vs Class

Use Struct when

- Small
- Immutable
- Represents one value
- Frequently allocated
- Short lifetime

Examples

- Point
- Vector
- Color
- DateOnly

Use Class when

- Large object
- Shared state
- Mutable
- Inheritance required
- Identity matters

13. Mutable Struct Problem

Bad

CSharp

```
public struct Point
```

```
{  
    public int X;  
    public int Y;  
}
```

Changing copies creates confusing bugs.

Prefer

CSharp

```
readonly struct Point  
{  
    public int X { get; }  
    public int Y { get; }  
}
```

Immutable structs are safer.

14. readonly struct

Benefits

- No accidental mutation
- Better compiler optimizations
- Reduced defensive copying

15. Large Struct Problem

Imagine

```
CSharp
struct HugeData
{
    // 40 fields...
}
```

Passing this repeatedly causes expensive copies.

Solution

```
CSharp
void Process(in HugeData data)
```

16. Equality

Value Types

Default compares values.

CSharp

```
bool result = (5 == 5); // Returns true
```

True

Reference Types

Default compares references.

CSharp

```
bool result = (obj1 == obj2); // Compares memory addresses
```

Unless equality is overridden.

17. Strings Are Special

String is a reference type.

But behaves like a value.

CSharp

```
string a = "Hello";  
string b = a;  
b += " World"; // Creates a new string; 'a' is unchanged
```

a remains unchanged.

Why?

Strings are immutable.

18. Nullable Types

CSharp

```
int? nullableInt = null; // Shorthand for Nullable<int>
```

A struct.

Not a class.

19. Generic Constraints

CSharp

```
where T : struct  
where T : class  
where T : unmanaged  
where T : notnull
```

These affect

- Allocation
- Boxing
- Performance
- API behavior

20. Record vs Record Struct

CSharp

```
record Person(string Name); // Reference type  
record struct Point(int X, int Y); // Value type
```

21. Performance Trade-offs

Scenario	Value Type	Reference Type
Small data	✓ Excellent	Good
Large objects	✗ Copy cost	✓ Better
Heap allocation	Usually avoided	Required
Garbage Collection	Minimal	Higher
Shared state	Poor	Excellent
Immutability	Recommended	Optional
Inheritance	No	Yes
Memory locality	Excellent	Can be fragmented

22. Common Mistakes

- ✗ Thinking value types always live on the stack.
- ✗ Thinking reference types store objects.
- ✗ Creating huge mutable structs.
- ✗ Excessive boxing.
- ✗ Assuming reference types are passed by reference.
- ✗ Forgetting `readonly struct`.
- ✗ Comparing reference types without overriding equality.

23. CLR Internal Working

When a value type is created:

- Memory is reserved where the variable lives.
- Bits representing the value are stored directly.
- Assignment copies every byte.

When a reference type is created:

- CLR allocates memory on the managed heap.
- Object header is created.
- Method table pointer is added.
- Fields are initialized.
- A reference (pointer-like value) is stored in the variable.
- The Garbage Collector later reclaims the object when no references remain.

24. Interview Questions

- Why are structs generally faster than classes?
- Do value types always live on the stack?
- What happens during boxing?
- Why can boxing hurt performance?
- When should you use `readonly struct`?

- Why is **string** a reference type but behaves like a value?
- Explain **ref**, **out**, and **in**.
- Why can large structs reduce performance?
- What is copied during assignment of a reference type?
- What is stored inside a reference variable?

Key Takeaways

- Value types store the **actual value**; reference types store a **reference** to an object.
- Assignment copies **values** for value types and **references** for reference types.
- Value types are **not guaranteed** to be on the stack, and reference types are **not guaranteed** to be only on the heap in the simplistic way often taught.
- Boxing allocates a new object on the heap and should be minimized in performance-critical code.
- Prefer **small, immutable (readonly) structs** and use **classes** for larger, shared, or identity-based objects.
- Understanding the CLR's memory model is more valuable than memorizing "stack vs heap" rules.

This topic forms the foundation for understanding **memory management, garbage collection, performance optimization, records, immutability, async behavior, generics**, and advanced .NET development.